

Autoresearcher as a Research Intern: Evidence-Gated Human–AI Collaboration for Scientific Workflow Automation

Anonymous authors

Paper under double-blind review

Abstract

Large language model agents are increasingly able to plan experiments, write code, run tests, and synthesize reports. Yet the dominant framing of fully autonomous “AI scientists” is misaligned with the practical needs of many human researchers: a scientist often does not need an unconstrained agent to discover an entire research program, but a reliable research intern that can rapidly turn a rough idea into auditable evidence, stop when evidence is weak, and escalate ambiguous decisions to the human scientist. We present **Autoresearcher**, a repository-centered human–AI research pipeline for small-scale scientific prototyping. Autoresearcher decomposes the research loop into role-resumed local coding agents for supervision, execution, review, and summarization; requires machine-readable decisions, plans, results, and reviews; enforces hard runtime and artifact-validity checks; and optionally escalates high-stakes decisions to a stronger ChatGPT-5.5-Pro checkpoint supervisor operating in a visible same-thread conversation. The core design principle is *evidence-gated co-autonomy*: the agent may propose and execute experiments, but durable repository artifacts, deterministic validators, review gates, and human/Pro checkpoints determine whether the project continues. We evaluate Autoresearcher through two live reinforcement-learning research projects. One produced scoped positive evidence for a soft terminal marginalization estimator and negative evidence for a tested low-rank auxiliary-goal variant. The other identified useful stochastic-TRL diagnostics but ultimately stopped after showing that several apparent improvements were explained by stronger baselines rather than a distinct stochastic-TRL mechanism. These case studies demonstrate that Autoresearcher can accelerate complex research workflows not only by finding positive signals, but by producing structured negative results, preventing unsupported scaling, and turning ambiguous automatic progress into explicit human decision points.

1 Introduction

Scientific progress often depends on a mundane but expensive loop: formulate a hypothesis, translate it into code, construct a minimal diagnostic, run the diagnostic, inspect results, decide whether to continue, and document what happened. For computational scientists and machine-learning researchers, much of this loop is not limited by imagination but by engineering bandwidth. A student or small lab may have many plausible ideas but too little time to build clean prototypes, run fair baselines, audit artifacts, and write down negative evidence.

Recent work on LLM-based scientific agents has made this bottleneck more visible. Systems such as The AI Scientist (Lu et al., 2024) and AI Scientist-v2 (Yamada et al., 2025) frame the goal as fully automated scientific discovery. Agent Laboratory explores LLM agents as research assistants (Schmidgall et al., 2025). MLAGentBench evaluates language agents on machine-learning experimentation tasks (Huang et al., 2023). Robin and SciAgents explore multi-agent scientific discovery workflows in specific domains (Ghareeb et al., 2025; Ghafarollahi & Buehler, 2024). These systems show that agentic research automation is

possible, but they also raise a practical question: what form of autonomy is most useful to a human scientist today?

We argue that many researchers need neither a passive coding assistant nor a fully autonomous scientist. They need a *research intern*: a system that can accept a rough human research idea, conduct small falsifiable prototypes, maintain a lab notebook, surface evidence, ask for guidance at important gates, and stop when the current direction is no longer justified. This paper presents Autoresearcher, a prototype of that paradigm.

2 Autoresearcher

Autoresearcher runs an iterative loop: supervisor, executor, reviewer, summarizer, and checkpoint policy. The scientist writes a charter specifying the hypothesis, metrics, scope, success criteria, and failure criteria. The local supervisor proposes exactly one next experiment or chooses pivot/stop/human review. The executor implements the plan and writes result artifacts. The reviewer audits evidence. A deterministic orchestrator validates outputs and decides whether the project may continue locally or must escalate to human/ChatGPT-Pro review.

Repository-centered memory. Autoresearcher treats files, not chat context, as memory. Each project has a charter, state file, plan directory, result directory, artifact directory, review directory, progress summaries, metric ledger, and Pro checkpoint packets. Role-resumed Codex sessions and the ChatGPT thread provide continuity, but every role is instructed that repository files are the source of truth.

Evidence-gated autonomy. The executor must write a result JSON containing the experiment id, status, claim tested, commands run, metrics, baseline metrics, artifact paths, interpretation, known failures, and next questions. The reviewer must return a verdict and say whether automatic continuation is allowed. The orchestrator validates JSON schemas and artifact paths before committing or proceeding.

Strong-model checkpoints. Autoresearcher optionally builds a compact review packet and sends it to a same-thread visible ChatGPT-5.5-Pro supervisor. This is used for cadence checkpoints, stop/pivot decisions, reviewer failures, timeouts, protected-file drift, and publication boundaries. The stronger model is therefore used as a senior checkpoint reviewer, not as a hidden API replacement.

3 Case study protocol

We evaluate Autoresearcher on two live projects proposed by the human scientist. These were open-ended reinforcement-learning research ideas with uncertain outcomes. We report whether the system could translate charters into executable diagnostics, create auditable artifacts, avoid unsupported scaling, and turn positive and negative evidence into explicit decisions.

4 Case study A: Reward-to-GCRL

The `reward_to_gcrl` project tests whether a soft successor-measure reward-to-goal conversion can preserve reward-to-goal equivalence while avoiding sparse high-variance terminal sampling. The system progressed through one-state variance diagnostics, CliffWalking equivalence checks, RiverSwim propagation, FourRooms vector successor-measure checks, and low-rank auxiliary-goal tests.

The final synthesis supports a scoped estimator claim: soft terminal marginalization removes terminal-sampling variance, preserves normalized-Q scaling in small audited tabular settings, and helps under adequate coverage. It also records negative auxiliary evidence: the tested rank-4 low-rank FourRooms auxiliary setup produced negative transfer, and

90 repair attempts did not match terminal-only performance. This is a successful research-
 91 intern outcome because the system separated supported estimator claims from unsupported
 92 auxiliary-representation claims and generated red-line claim boundaries before external
 93 use.

94 5 Case study B: Stochastic transitive RL

95 The `sto_trl` project tests whether calibrated stochastic successor distances plus TRL-style
 96 divide-and-conquer path relaxation can improve long-horizon offline goal-conditioned
 97 RL under stochastic dynamics. The system built exact-DP tabular diagnostics and tested
 98 raw TRL, log TRL, successor-distance variants, posterior/transitive variants, and partial-
 99 observation history/context pivots.

100 The project produced useful findings: raw deterministic-style TRL can overestimate lucky
 101 stochastic paths; log-space TRL helps long-horizon propagation relative to MC-only under
 102 censored labels; successor-distance variants did not add distinct value beyond TRL-log;
 103 posterior/transitive variants were explained by fair model-DP baselines; and in aliased
 104 POMDPs, history/context helps, but history-model DP explains the gains. The final stronger-
 105 model decision stopped the loop, preventing a misleading claim of a distinct stochastic-TRL
 106 algorithmic advantage.

107 6 Discussion

108 Autoresearcher accelerated research by converting rough ideas into executable diagnostics,
 109 producing decisions rather than only raw outputs, and enforcing claim boundaries. Its most
 110 important behavior may be epistemic discipline: it produced positive signals when justified,
 111 but also stopped weak directions and preserved negative evidence.

112 Compared with fully autonomous AI-scientist systems, Autoresearcher is intentionally less
 113 autonomous and more useful as an everyday scientific collaborator. The scientist retains
 114 authority over goals, pivots, scaling, and publication. The agent handles implementation,
 115 bookkeeping, and evidence synthesis. This is a practical human-AI collaboration paradigm
 116 for students and small labs.

117 7 Limitations

118 This is an early prototype. We report two case studies, not a controlled productivity study.
 119 Both are reinforcement-learning-style computational projects. The ChatGPT-Pro bridge is
 120 useful but brittle, with possible blockers from login state, browser bridge failure, parse
 121 errors, model availability, or rate limits. The current implementation is optimized for small-
 122 scale prototypes and intentionally blocks larger training or benchmark-scale validation
 123 without review. Human review remains necessary before external scientific claims.

124 8 Conclusion

125 Autoresearcher demonstrates a practical human-AI collaboration paradigm for scientific
 126 workflows: an AI research intern that can execute small experiments, maintain an auditable
 127 lab notebook, synthesize evidence, and escalate important decisions. Across two live
 128 research projects, the system produced scoped positive evidence, useful negative results,
 129 and stopping decisions that prevented unsupported scaling. Evidence-gated co-autonomy
 130 is a promising path toward LLM systems that genuinely empower human scientists.

131 References

132 Alireza Ghafarollahi and Markus J. Buehler. SciAgents: Automating scientific discovery
 133 through multi-agent intelligent graph reasoning, 2024. URL [https://arxiv.org/](https://arxiv.org/abs/2409.05556)
 134 [abs/2409.05556](https://arxiv.org/abs/2409.05556).

- 135 Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J.
136 Szostkiewicz, Jon M. Laurent, Muhammed T. Razzak, Andrew D. White, Michaela M.
137 Hinks, and Samuel G. Rodriques. Robin: A multi-agent system for automating scientific
138 discovery, 2025. URL <https://arxiv.org/abs/2505.13400>.
- 139 Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAGentBench: Evaluating
140 language agents on machine learning experimentation, 2023. URL <https://arxiv.org/abs/2310.03302>.
- 142 Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI
143 Scientist: Towards fully automated open-ended scientific discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- 145 Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang
146 Liu, Zicheng Liu, and Emad Barsoum. Agent Laboratory: Using llm agents as research
147 assistants, 2025. URL <https://arxiv.org/abs/2501.04227>.
- 148 Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff
149 Clune, and David Ha. The AI Scientist-v2: Workshop-level automated scientific discovery
150 via agentic tree search, 2025. URL <https://arxiv.org/abs/2504.08066>.